System Verification and Validation Plan for Software Engineering

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

November 3, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 3

3 Plan **3**
- 3.1 Verification and Validation Team . . . 3
- 3.2 SRS Verification . . . 3
- 3.3 Design Verification . . . 4
- 3.4 Verification and Validation Plan Verification . . . 5
- 3.5 Implementation Verification . . . 8
- 3.6 Automated Testing and Verification Tools . . . 9
- 3.7 Software Validation . . . 9

4 System Tests **10**
- 4.1 Tests for Functional Requirements . . . 10
  - 4.1.1 Outline Intake and Extraction . . . 10
  - 4.1.2 Calendar and Conflict Detection . . . 14
  - 4.1.3 Scheduling Suggestions . . . 16
  - 4.1.4 Compliance Checking . . . 17
  - 4.1.5 Access and Export . . . 18
- 4.2 Tests for Nonfunctional Requirements . . . 19
  - 4.2.1 Course Outline Processing . . . 19
  - 4.2.2 Calendar and Web Application . . . 21
  - 4.2.3 Security, Reliability, and Deployment . . . 23

5 Unit Test Description **25**
- 5.1 Unit Testing Scope . . . 25
- 5.2 Tests for Functional Requirements . . . 25
  - 5.2.1 Module ? . . . 25
  - 5.2.2 Module ? . . . 26
- 5.3 Tests for Nonfunctional Requirements . . . 26
  - 5.3.1 Module ? . . . 26

5.3.2 Module ? 27
5.4 Traceability Between Test Cases and Modules 27

6 Appendix **29**
6.1 Symbolic Parameters 29
6.2 Usability Survey Questions? 29

7 Appendix — Generative AI Use Disclosure **30**
7.1 AI Tools Used 30
7.2 How and Where Used 30
7.3 Verification of AI-Generated Content 30

8 Appendix — Reflection **31**

List of Tables

1 VnV Plan Verification Activities 5
2 Seeded Faults and the Tests That Detect Them 7
[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
ASSURE	Accreditation and Scheduling Support for Undergraduate Review and Evaluation
CAS	Department of Computing and Software
CEAB	Canadian Engineering Accreditation Board, the organization responsible for accrediting undergraduate engineering programs in Canada.
FMEA	Failure Mode and Effect Analysis
LLM	Large Language Model
PDF	Portable Document Format
PoC	Proof of Concept
SRS	Software Requirements Specification (Mehta et al., 2026b)
T	Test
V&V	Verification and Validation

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

The software under test is ASSURE (Accreditation and Scheduling Support for Undergraduate Review and Evaluation), which is a course-outline review system for the Department of Computing and Software (CAS) at McMaster University.

ASSURE accepts a batch of course outlines in PDF (Word and LaTeX document support will be a future feature). A replaceable (black box) locally hosted open-weight large language model (LLM) extracts each assessment's name, date, weight, and learning outcomes, and it either cites the passage each value came from or reports that it cannot find the value. The system checks the extracted information against McMaster University's Undergraduate Course Management Policy (McMaster University Senate, 2025), academic-calendar restrictions, and the Canadian Engineering Accreditation Board's (CEAB) Accreditation Criteria and Procedures (Canadian Engineering Accreditation Board, 2025), and flags missing, incomplete, or non-compliant information for review. Its dashboard supports filtering by course, year, and program. It also displays term completeness, workload or scheduling conflicts, and allows users to override or correct extracted values. The system also stores the reviewed data for export, while its shared calendar presents assessment deadlines across CAS courses and highlights items that need attention.

2.2 Objectives

The primary objective of the Verification and Validation (V&V) process is to build confidence that the CAS system functions correctly and reliably, as well as ensuring that the information it produces is accurate and traceable to the original course outlines. Emphasis will be placed on verifying the accuracy of document processing and information extraction, including outputs generated through the external LLM. The system must also reliably support its core functionality, including the management and presentation of extracted

information, to ensure that CAS administrators and the associate chairs can use the results to review course outlines against CEAB requirements and McMaster policies.

Given resource and time constraints, certain attributes will receive less attention during the V&V process. For example, usability studies involving a large group of users and performance testing under high workloads will be outside the scope of this project. Basic usability and performance checks will still be conducted, but testing efforts will prioritize correctness, reliability, and traceability due to their greater impact and importance on the system's core purpose. Additionally, the internal implementation of external dependencies, such as third-party document processing libraries and the LLM, will not be independently verified by the team. Instead, these will be treated as externally maintained components, with a focus on integration testing and evaluating the accuracy of their outputs.

2.3 Custom Documents

The project includes two approved Custom Documents that support the design process and end-user adoption:

1. **Fall Term CD: Wireframe Report.** The wireframe report will document the proposed administrative dashboard, including the shared assessment calendar, course filtering tools, and scheduling recommendation displays. The wireframes will be reviewed with the project clients and stakeholders to confirm that the navigation, component interactions, and workflows reflect their needs before front-end implementation begins. Feedback from this review will be used to revise the interface design.

2. **Winter Term CD: User Manual.** The user manual will provide instructors and CAS administrative staff with step-by-step instructions for uploading course outlines, filtering the centralized assessment calendar, exporting database records, and reviewing policy compliance flags and scheduling recommendations. The manual will be checked against the implemented system and reviewed for completeness, clarity, and whether users can follow its procedures to complete the documented tasks.

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

The SRS will be verified through a combination of structured internal reviews, as well as receiving feedback from primary stakeholders (CAS administrators, associate chairs). Team members will review the SRS to assess the completeness, clarity, consistency, feasibility, and verifiability of the requirements. To ensure a systematic review, the team will use the following checklist for verification:

- Completeness: Verify that all required system functions and relevant constraints are documented.
- Clarity: Ensure that each requirement is clearly stated and free of ambiguity.

- Consistency: Verify that the requirements do not conflict with one another and use consistent terminology.
- Feasibility: Ensure that each requirement can realistically be implemented within the project's technical constraints.
- Verifiability: Ensure that each requirement can be tested or verified using defined criteria.
- Traceability: Verify that each requirement can be linked to its relevant stakeholder needs, as well as identified risks in the Hazard Analysis.

The SRS will be reviewed with the project's TA (Lucas Dutton) and primary stakeholders (Matt Vonk, Connie Carrabs, Spencer Smith, Sebastien Mosser) during the scheduled biweekly meetings. Feedback on the requirements will be to determine whether the functionality accurately reflects stakeholder needs and project expectations. The team will encourage reviewers to identify any missing requirements or unclear descriptions, and any issues identified during reviews will be documented and tracked using GitHub Issues. The team will then address the feedback, update the SRS as needed, and review the changes to ensure that identified issues have been resolved.

3.3 Design Verification

Methods we will use for design verification.

- **Testing.** We will run design checks against the planned behaviour. Examples include outline intake, extraction with a source citation, confirmation before use, restricted-period and weight flags without the large language model (LLM), and rejecting a bad file. Each check will name the SRS or Hazard Analysis requirement it covers, and will state a concrete expected result.
- **Analysis.** We will review the design on paper. We will revisit the Failure Mode and Effect Analysis (FMEA) in the Hazard Analysis and ask whether each mitigation still has a place in the design. If something is deferred, we will point to the Hazard Analysis roadmap.
- **Inspection.** A teammate who did not write the design will use a checklist covering the system components C1 to C11 from the Hazard

Analysis. Our supervisors will inspect the design in a meeting when needed. Classmate peer reviewers will use the same checklist and leave comments as GitHub issues.

- **Similarity.** We will compare a new design to an earlier accepted design choice from the Development Plan, such as a replaceable LLM interface. For example, switching the LLM from Copilot to Gemma. If we change that choice, we will explain why the change still fits the SRS and Hazard Analysis.

Results and pass/fail data. For each design check, we will record which design part we looked at, which SRS or Hazard Analysis requirement we used as the expected result, what we observed, and whether it passed or failed.

Non-conformances and deviations. If a design check fails, we will open a GitHub issue that names the design part and what it failed against. The issue will also say how we fixed it, or why we are allowing a deviation for now, before we treat the design as ready to implement.

3.4 Verification and Validation Plan Verification

The VnV plan is itself verified before it is submitted, and again before the final version, once the unit test sections are complete. Five activities are used, summarized in Table 1. Each one produces a record, so the team can show what was checked and what was changed as a result.

Table 1: VnV Plan Verification Activities

Activity	**Done by**	**Evidence recorded**
Checklist review	A member who did not write the section	Pull request review comments and approval
Traceability and coverage check	Traceability table owner	Gaps found, opened as GitHub issues
Test dry run	A member who did not write the tests	Tests checked and tests rewritten
Seeded fault review	Whole team	Table 2, with any test added

Activity	Done by	Evidence recorded
External review	Team 22, the TA, and the supervisors	Peer review issues, TA feedback issues, and supervisor meeting notes

Checklist review of each section. Every section is added through a pull request, and a team member who did not write it reviews it against the course VnV checklist. The reviewer checks, in particular, that each test has specific input, an explicit expected output, and a derivation for that output, and that symbolic constants are used instead of literal values. Each checklist item that is not met becomes a review comment, and the pull request is merged only after every comment is resolved or answered with a reason.

Traceability and coverage check. The team member responsible for the traceability tables checks the whole plan before it is submitted:

- every requirement in the SRS (Mehta et al., 2026b) and every safety and security requirement in the Hazard Analysis (Mehta et al., 2026a) appears in the traceability table in Section 4.3;
- every requirement meets the coverage rule of at least two verification methods;
- every test identifier in the traceability tables is defined in the system or unit test sections, and every defined test is traced; and
- every module is traced to at least one unit test and one system test.

Each gap found is opened as a GitHub issue and assigned to the author of the affected section.

Test dry run. For each area of system tests, a team member who did not write the tests selects three of them and walks through each one using only the information in this document. The test passes the dry run if the member can name the input files, carry out the steps, and state the expected output without asking the author. A test that fails the dry run is rewritten before submission. This checks that someone outside the team could run the tests once the product is built.

Seeded fault review. This is a form of mutation testing applied to the plan. The team lists faults that a plausible implementation could contain, and for each fault identifies the planned test that would fail because of it. Table 2 gives the initial list. A fault that no planned test would detect shows a missing test, and a test is added for it. The review is held in a team meeting, and the target is that every seeded fault is detected by at least one planned test.

Table 2: Seeded Faults and the Tests That Detect Them

Seeded fault	**Detected by**
Weights that total 90% pass the weight check.	ST_Comp_Policy
A missing required outline section is not reported.	ST_Comp_Policy
A date marked TBD is stored as a date instead of being flagged as missing.	ST_Extract_Flag
A blank weight is filled with a guessed value.	ST_Extract_Flag
An extracted value is stored without a citation to its source passage.	ST_Extract_Assessment
An unconfirmed assessment appears in the calendar.	ST_Extract_Review
An assessment dated in reading week is not reported.	ST_Work_Policy
A suggested date falls in a restricted period.	ST_Suggest_Generation
An assessment date changes without the instructor's approval.	ST_Suggest_NoAuto
A course missing from the master course sheet is shown with no findings.	ST_Comp_Grad
An instructor can change another instructor's course.	ST_Access_Roles
A file of 5 MB or larger is accepted for processing.	ST_Intake_Data
A protected page opens without logging in.	ST_Access_Login
An instructor can export the CSV file.	ST_Export_CSV
A failed upload deletes or changes stored records.	NFR-REL-01

Seeded fault	Detected by
A credential appears in the deployment configuration or logs.	NFR-SEC-01

External review. Three groups outside the team review the plan:

- **Peer review team:** Team 22, the primary reviewer, reviews the plan and opens GitHub issues using the repository's peer review template. The team answers each issue by fixing it in a linked pull request, or by closing it with a stated reason.
- **Teaching assistant:** The team asks the TA the following three questions during during the informal presentation, the team asks the TA three questions. Is the set of sample outlines large enough to show that the accuracy targets in NFR1 are met? Are the planned reviews, inspections, and walkthroughs enough to support the dynamic tests? Can the plan be completed in the time available? The TA's answers are recorded as GitHub issues.
- **Supervisors:** At a supervisor meeting, the team presents the system test areas and the sample outlines used as test input, and asks Matt Vonk and Connie Carrabs whether the outlines represent CAS courses and whether the policy rules being tested are the ones the department applies. Their answers are recorded in the meeting notes, and any change becomes an issue.

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

3.6 Automated Testing and Verification Tools

The primary coding language for the system will be Python, so the team will use the following automated testing and verification tools to support code quality, identify potential defects, and evaluate test coverage throughout development:

- Pytest: A Python testing framework that will be used to develop and execute automated unit tests for individual functions and modules.
- Pytest-cov: A code coverage tool that will be used alongside Pytest to measure the proportion of source code executed during testing. Coverage reports will be reviewed to identify parts of code that may require additional testing.
- Ruff: A Python linter and static analysis tool that will be used to identify potential errors, enforce consistent coding standards, and improve code quality.
- GitHub Actions: A continuous integration tool that will automatically run the project's test suite and code quality checks when changes are pushed to the repository.

Special attention will be paid to the components responsible for the fundamental functions of the system, such as document processing, information extraction, and data validation. The team will review automated test results to identify and resolve issues before integrating changes into the main codebase. The specific configuration and use of these tools may be adjusted as development and implementation progresses.

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should

plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

This section describes the system-level tests for ASSURE. Section 4.1 covers the functional requirements in the SRS. There is one system test for each functional requirement, labeled ST-FR1 through ST-FR17. The tests are grouped the same way as the SRS: outline intake and extraction, calendar and conflict detection, scheduling suggestions, compliance checking, and access and export.

4.1 Tests for Functional Requirements

Each test below checks one functional requirement end to end through the running system. The inputs (files, dates, accounts, and expected values) are stated in each test.

There is one system test for each functional requirement (FR1–FR17). Some tests check more than one input case (for example, valid and invalid uploads in ST-FR1).

For this document version, outline intake is **PDF only**, and other formats will be supported later. For additional formats, new intake tests will be added. The max outline size is 5 MB, as defined in `SSR_AcceptedFiles`. Symbolic constants used in the tests below (cluster window, cluster threshold, min alternatives, weight total) are from SRS Appendix C.

4.1.1 Outline Intake and Extraction

These tests cover SRS requirements FR1 to FR6.

1. ST-FR1: Outline intake (FR1)

   Type: Functional, Dynamic, Manual

Initial State: A CAS administrator is logged in. No outline is stored for the selected course in the current term.

Input/Condition: (a) A valid PDF course outline for that course, smaller than the max outline size (5 MB); (b) the same outline content saved in a non-PDF file format; (c) a PDF of the same outline larger than 5 MB.

Output/Result: (a) The system accepts the upload, stores the outline for that course, and shows a success message. (b) and (c) The system rejects the file before processing and does not store an outline.

Test Case Derivation: FR1 requires PDF outline intake from a CAS administrator or instructor. SSR_AcceptedFiles requires rejecting non-PDF and oversized files before processing.

How test will be performed: The tester uploads (a)–(c) through the web interface and records whether each file is accepted or rejected. A second pass with an instructor account confirms that the same valid PDF upload path is available to instructors.

2. ST-FR2: Extract assessments (FR2)

   Type: Functional, Dynamic, Manual

   Initial State: A valid PDF course outline for the selected course is stored. Extraction has not been confirmed.

   Input/Condition: The stored PDF, which contains Midterm 1 (weight 30%, dated 2026-02-10), Assignment 1 (blank weight, dated 2026-01-20), and Final Exam (weight 50%, date TBD).

   Output/Result: The system shows three assessments with those names, types, dates, and weights. Each extracted value includes a citation to its source passage in the outline. The Final Exam date and Assignment 1 weight remain empty or marked missing; the system does not invent values for them.

   Test Case Derivation: FR2 requires extraction of each assessment's name, type, date, and weight. The blank weight and TBD date also set up FR4.

   How test will be performed: The tester runs extraction and compares the structured result to a handwritten answer key for that PDF.

3. ST-FR3: Extract learning outcomes (FR3)

   Type: Functional, Dynamic, Manual

   Initial State: A valid PDF course outline for the selected course is stored. The outline includes at least three learning outcomes that identify graduate attributes by number, by name, and by description only.

   Input/Condition: The stored outline, which contains those three learning outcomes: one with GA numbers beside it, one that names the attribute in words, and one that only describes the attribute.

   Output/Result: All three outcomes appear with their text. The system records the graduate attributes addressed in each of the number, name, and description cases. Each extracted value cites its source passage.

   Test Case Derivation: FR3 requires extraction of learning outcomes and the graduate attributes each addresses, whether listed by number, name, or description.

   How test will be performed: The tester runs extraction and compares the outcomes and GA mappings to the handwritten answer key.

4. ST-FR4: Flag missing fields (FR4)

   Type: Functional, Dynamic, Manual

   Initial State: A valid PDF course outline has been extracted into structured assessments, including Final Exam with date TBD and Assignment 1 with a blank weight.

   Input/Condition: The extracted assessment list that includes Final Exam (date TBD) and Assignment 1 (blank weight).

   Output/Result: The system flags the TBD date and the blank weight as missing fields. It does not fill either field with a guessed value.

   Test Case Derivation: FR4 requires flagging of any extracted field that is missing, including a date marked TBD.

   How test will be performed: The tester inspects the extraction review screen and compares the missing-field flags to the handwritten answer key.

5. ST-FR5: Confirm extracted data (FR5)

   Type: Functional, Dynamic, Manual

   Initial State: Extracted assessment data for the selected course is shown on the review screen and has not been confirmed.

   Input/Condition: The reviewer confirms all assessments except the Final Exam, which remains unconfirmed.

   Output/Result: Confirmed assessments appear in calendar, conflict, and compliance views. The unconfirmed Final Exam does not appear in those views until it is confirmed.

   Test Case Derivation: FR5 requires user review and confirmation before extracted data is used for calendar, conflict detection, or compliance checking.

   How test will be performed: The tester opens the calendar and compliance views before confirmation, after confirming all assessments except the Final Exam, and after confirming the Final Exam.

6. ST-FR6: Tentative dates (FR6)

   Type: Functional, Dynamic, Manual

   Initial State: An instructor for a course is logged in. An assessment for that course has a missing date.

   Input/Condition: The instructor enters the tentative date 2026-02-17 for that assessment, then updates it to 2026-02-24.

   Output/Result: After each change, the stored date is shown as tentative. The final stored date is 2026-02-24. An instructor for a different course cannot change this date.

   Test Case Derivation: FR6 allows an instructor to enter and update tentative assessment dates for their own courses.

   How test will be performed: The course instructor sets and updates the date. A second instructor account for a different course is used to verify that the date cannot be edited.

4.1.2 Calendar and Conflict Detection

These tests cover SRS requirements FR7 to FR10.

1. ST-FR7: Shared calendar (FR7)

   Type: Functional, Dynamic, Manual

   Initial State: Confirmed and tentative dates exist for at least two courses in the same program and year level.

   Input/Condition: (a) Filter the shared calendar by a selected program and year level; (b) then filter by one selected course alone.

   Output/Result: Under (a), confirmed and tentative dates for matching courses appear, tentative dates are visually marked, and courses from other programs do not appear. Under (b), only that course's events appear.

   Test Case Derivation: FR7 requires a shared calendar filtered by course, year, and program, with tentative dates marked.

   How test will be performed: The tester applies each filter and compares the visible events to the stored assessments.

2. ST-FR8: Workload cluster (FR8)

   Type: Functional, Dynamic, Automatic

   Initial State: Confirmed assessments for one cohort are loaded so that four assessments fall inside one 7-day cluster window. A second cohort has only two assessments in that same window. The cluster threshold is set to 3.

   Input/Condition: The loaded cohort assessments and the cluster threshold of 3.

   Output/Result: The system flags the first cohort's window as a workload cluster. The second cohort is not flagged for that window.

   Test Case Derivation: FR8 requires identifying cluster windows where a cohort has more assessments than the configurable threshold.

   How test will be performed: A scripted system test loads the fixture data and asserts which windows are flagged.

3. ST-FR9: Restricted periods (FR9)

   Type: Functional, Dynamic, Automatic

   Initial State: The academic calendar for the term includes a reading week (recess) and a final examination or test-ban period (`SSR_RestrictedDates`).

   Input/Condition: One confirmed assessment dated inside reading week, one dated inside the exam or test-ban period, and one dated outside all restricted periods.

   Output/Result: The reading-week and exam or test-ban assessments are flagged as restricted. The unrestricted assessment is not flagged for this check.

   Test Case Derivation: FR9 requires identifying assessments scheduled in periods restricted by McMaster policy (for example, final exam periods). H6-4 calls for covering each restricted-period type used in the calendar.

   How test will be performed: A scripted system test runs conflict detection and checks the restricted-period findings against the loaded academic calendar.

4. ST-FR10: Re-analyze conflicts (FR10)

   Type: Functional, Dynamic, Automatic

   Initial State: Conflict results are stored that flag a workload cluster for one cohort (four assessments inside a 7-day window with a cluster threshold of 3).

   Input/Condition: An instructor changes one assessment date so that the cluster no longer exceeds the cluster threshold of 3.

   Output/Result: Conflict detection runs again for the affected courses and cohorts. The previous cluster flag is cleared.

   Test Case Derivation: FR10 requires re-running conflict detection when an assessment date is added or changed.

   How test will be performed: A scripted system test updates the date and asserts the refreshed conflict results.

4.1.3 Scheduling Suggestions

These tests cover SRS requirements FR11 and FR12.

1. ST-FR11: Suggest alternative dates (FR11)

   Type: Functional, Dynamic, Manual

   Initial State: One assessment is in a detected conflict, and another has a TBD date. The instructor for that course is logged in.

   Input/Condition: (a) The instructor explicitly requests suggestions for both assessments; (b) the instructor does not request suggestions.

   Output/Result: Under (a), the system offers at least two feasible dates for the conflicted assessment and suggestions for the TBD assessment, and no suggested date falls in a restricted period. Under (b), no suggestions are created.

   Test Case Derivation: FR11 requires alternative dates on explicit instructor request for conflicted or TBD assessments.

   How test will be performed: The tester requests suggestions and records the dates and explanations shown. A second run without a request confirms that no suggestions appear automatically.

2. ST-FR12: No automatic date change (FR12)

   Type: Functional, Dynamic, Manual

   Initial State: Alternative date suggestions for a conflicted or TBD assessment are pending. The responsible instructor is logged in.

   Input/Condition: (a) The instructor rejects all suggestions; (b) in a second trial, the instructor accepts one suggestion.

   Output/Result: After (a), the assessment date is unchanged. After (b), the stored date is the accepted suggestion and changes only because the instructor approved it.

   Test Case Derivation: FR12 requires that an assessment date does not change without the responsible instructor's approval.

   How test will be performed: The tester rejects all suggestions, then accepts one suggestion in a second trial, and checks the stored date after each action.

4.1.4 Compliance Checking

These tests cover SRS requirements FR13 and FR14.

1. ST-FR13: Policy compliance (FR13)

   Type: Functional, Dynamic, Manual

   Initial State: A confirmed outline has assessment weights that sum to 90% and omits one required outline element named in the answer key (from the Undergraduate Course Management Policy rules encoded for FR13). A second confirmed outline that satisfies the same rules is available as a clean control.

   Input/Condition: (a) The defective confirmed outline and the encoded McMaster course-outline policy rules; (b) the clean control outline and the same rules.

   Output/Result: Under (a), the system reports a finding for the omitted required element and a finding that weights do not total 100%, and it lists which rules were checked (`SSR_CheckCoverage`). Under (b), the clean control produces no false weight-total failure.

   Test Case Derivation: FR13 requires checking outlines against McMaster policy rules, including required sections and weights totalling 100%. Related encoded rules include a maximum single weight of 75% and early-feedback minima.

   How test will be performed: The tester runs compliance on the defective outline and on the clean control outline and compares the findings to the answer key.

2. ST-FR14: Graduate attribute check (FR14)

   Type: Functional, Dynamic, Manual

   Initial State: The master course sheet expects GAs 1, 2, and 5 for the selected course. The confirmed outline for that course lists GAs 1 and 2 only. A second course code has no master-sheet row.

   Input/Condition: (a) The confirmed learning-outcome GAs and the master course sheet row for the selected course; (b) the second course code with no master-sheet row.

Output/Result: Under (a), the system flags GA 5 as missing or inconsistent. Under (b), the course absent from the master sheet is reported as not checked, not as having no findings.

Test Case Derivation: FR14 requires comparing outline GA numbers with the master course sheet and flagging missing or inconsistent values. `SSR_CheckCoverage` / H5-5 require the not-checked wording.

How test will be performed: The tester runs the GA check on both courses and records the findings text.

4.1.5 Access and Export

These tests cover SRS requirements FR15 to FR17.

1. ST-FR15: Login required (FR15)

   Type: Functional, Dynamic, Manual

   Initial State: The tester is logged out.

   Input/Condition: A request to open the dashboard or a course page while logged out.

   Output/Result: The system denies access and prompts for login. After a valid login, the same page is reachable.

   Test Case Derivation: FR15 requires users to log in before they can view or change any data.

   How test will be performed: The tester opens protected URLs while logged out, then logs in and retries the same URLs.

2. ST-FR16: Role-based access (FR16)

   Type: Functional, Dynamic, Manual

   Initial State: An administrator account and an instructor account exist. The instructor owns only one course.

   Input/Condition: (a) The instructor attempts to edit another instructor's course; (b) the administrator edits that same course.

   Output/Result: Under (a), the instructor's change is denied. Under (b), the administrator's change succeeds.

Test Case Derivation: FR16 allows administrators to manage all courses and limits instructors to their own courses.

How test will be performed: The tester repeats the edit attempt with each role and records whether the change is accepted or denied.

3. ST-FR17: CSV export (FR17)

   Type: Functional, Dynamic, Manual

   Initial State: Confirmed assessment data exists for at least one course. An administrator account and an instructor account exist.

   Input/Condition: (a) The administrator requests a CSV export of extracted assessment data; (b) the instructor attempts the same export.

   Output/Result: Under (a), a CSV downloads that contains the extracted assessment fields for the selected scope. Under (b), the instructor cannot run the export.

   Test Case Derivation: FR17 allows an administrator to export extracted assessment data as CSV.

   How test will be performed: The tester exports as administrator and opens the file, then confirms that the export control is unavailable to an instructor.

4.2 Tests for Nonfunctional Requirements

The following tests assess the system-wide nonfunctional requirements. Accuracy measurements use a manually prepared answer key for a representative set of course outlines. Performance measurements will record the test environment, document characteristics, workload, and response times.

4.2.1 Course Outline Processing

1. **NFR-ACC-01: Extraction accuracy**

   Type: Dynamic, automatic

   Initial State: The course-outline processing service is available and the representative test dataset and manually prepared answer key are stored.

Input/Condition: Submit each test outline for processing.

Output/Result: The final structured output contains the expected assessment dates, assessment weights, and learning outcomes with at least 95% accuracy for each field type. At least 95% of predefined missing, unclear, or incomplete fields are flagged for human review.

How test will be performed: Run the extraction pipeline on every outline, compare the confirmed structured output with the answer key, and calculate precision/recall or an equivalent field-level accuracy for each field type. Calculate missing-field recall as the proportion of known missing or incomplete fields that were flagged. Record the results and compare them with the targets in the Accuracy NFR.

2. **NFR-ACC-03: Policy and CEAB compliance accuracy**

   Type: Dynamic, automatic

   Initial State: The compliance-checking service is available and the representative test dataset has a manually prepared answer key identifying all predefined policy and CEAB compliance issues.

   Input/Condition: Submit each test outline for compliance checking.

   Output/Result: The system identifies at least 95% of the predefined policy and CEAB compliance issues and reports each identified issue in the verification results.

   How test will be performed: Run the compliance checker on every outline, compare the reported issues with the answer key, and calculate recall as the number of correctly identified predefined issues divided by the total number of predefined issues. Record false positives separately and compare the recall with the 95% target in the Accuracy NFR.

3. **NFR-PERF-01: Outline processing time**

   Type: Dynamic, automatic

   Initial State: The processing service is available and the test environment has been recorded. A supported PDF course outline of no more than 30 pages is available.

Input/Condition: Submit a representative set of supported outlines, including documents with different layouts and extraction sizes.

Output/Result: At least 90% of successful processing runs complete within 60 seconds. Failures and delays caused by unavailable external dependencies are recorded separately.

How test will be performed: Run each outline through the complete document processing and information-extraction workflow, measure elapsed time from submission to structured output, and report the percentile and individual results against the 60-second target in the Performance NFR.

4.2.2 Calendar and Web Application

1. **NFR-PERF-02: Calendar response time**

   Type: Dynamic, automatic

   Initial State: The database contains the representative processed assessment dataset, the web application is running in the recorded test environment, and the user is authenticated.

   Input/Condition: Issue calendar-load requests using representative filters for course, year, and program under the expected concurrent workload.

   Output/Result: At least 95% of requests display the previously processed calendar data within 3 seconds.

   How test will be performed: Generate repeated calendar-load requests with the defined filters, measure the elapsed time until the calendar data is displayed, and report the response-time distribution and the percentage meeting the 3-second target in the Performance NFR.

2. **NFR-USE-01: Core task completion**

   Type: Dynamic, manual

   Initial State: Representative CAS administrators and course instructors who were not involved in implementation and do not have programming or machine learning knowledge are provided with access to the deployed system and the task scenarios.

Input/Condition: Each participant attempts the designated core tasks: uploading an outline, reviewing extracted information, viewing and filtering the calendar, interpreting a flagged issue, and exporting available data, without programming or machine learning assistance.

Output/Result: At least 80% of participants complete each designated core task without assistance or technical knowledge. Any usability issues, errors, requests for help, or evidence of programming or machine learning assistance are recorded.

How test will be performed: Observe each participant using the system without providing task-specific or technical assistance, record task completion and time, and collect post-task feedback about ease of use. Calculate the completion rate for every task and verify that the results demonstrate use by non-technical participants. Compare the results with the Usability NFR.

3. **NFR-PORT-01: Hosting and browser portability**

Type: Dynamic, automatic and manual

Initial State: The system is deployed in the designated McMaster University hosting environment. The deployed web application and its core workflows are available on supported desktop operating systems in Google Chrome 120 or later, Mozilla Firefox 121 or later, and Safari 17 or later.

Input/Condition: Run the authentication, outline submission, information review, calendar viewing, and export workflows in each supported browser and on the designated hosting environment.

Output/Result: The system is usable in the designated McMaster University hosting environment and the core workflows complete successfully in each supported browser. Automated accessibility checks report no unresolved high-severity issues against the applicable AODA and WCAG 2.0 Level AA criteria.

How test will be performed: Run an automated accessibility scan, then manually complete the core workflows in every supported browser and on the designated hosting environment. Use keyboard navigation and a screen-reader check, and record the browser, operating system, host-

ing environment, applicable WCAG success criteria, results, and any deviations against the Portability and Accessibility NFRs.

4.2.3 Security, Reliability, and Deployment

1. **NFR-SEC-01: Access control and data protection**

   Type: Dynamic, automatic and manual

   Initial State: Test accounts exist for each supported role, including an unauthenticated user, a professor, and a CAS administrator. Test data includes records that each role should and should not be able to access.

   Input/Condition: Attempt to access protected pages and API endpoints without authentication, with the wrong role, and with valid permissions. Inspect deployment configuration and logs for exposed credentials or sensitive configuration values.

   Output/Result: Unauthenticated and unauthorized requests are rejected. Permitted users can perform only the actions assigned to their roles, and credentials and sensitive configuration values are not exposed.

   How test will be performed: Execute the access-control test cases for each role, verify both the user interface and direct endpoint responses, and inspect configuration handling. Record every unexpected access or exposure and compare the results with the Security NFR.

2. **NFR-REL-01: Invalid input and dependency failure recovery**

   Type: Dynamic, automatic

   Initial State: The database contains valid course, assessment, and calendar records. A submitted outline is retained by the system.

   Input/Condition: Submit an unsupported, malformed, oversized, incomplete, or unreadable outline, and separately make the extraction or recommendation service unavailable during processing.

   Output/Result: The system reports an actionable error without crashing, corrupting or deleting existing records, and retains the submitted outline for retry. While the external service is unavailable, users can

still view and edit stored information, view the calendar, and export available data.

How test will be performed: Run each invalid-input and dependency-failure scenario, verify the response and stored data before and after the failure, and retry processing after restoring the dependency. Record whether each predefined case meets the Reliability NFR.

3. **NFR-DEP-01: Deployment and maintainability**

   Type: Static and dynamic, manual

   Initial State: A clean designated McMaster University hosting environment and the documented installation and deployment instructions are available to a maintainer who did not perform the original setup.

   Input/Condition: Deploy the system by following the documentation. Verify that the documentation identifies the required software, configuration values, environment variables, database setup, and service startup steps. Then implement representative isolated changes to document processing, scheduling, or compliance rules.

   Output/Result: The maintainer can deploy the system to the designated McMaster University hosting environment and complete the core workflow without undocumented steps. The documentation contains all required setup information. The representative changes do not require unrelated components to be rewritten, and the effort for each change is recorded relative to the original implementation effort for the affected component.

   How test will be performed: Give the documentation and clean hosting environment to a new maintainer without providing undocumented guidance. Observe the maintainer follow the installation and deployment instructions, execute the core workflow, and record any missing or ambiguous steps. Review the change diffs and effort records with the team, then document the results against the Installability, Deployability, and Maintainability NFRs.

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[Explain what is and is not in unit-test scope once the design is available. —SS]

5.2 Tests for Functional Requirements

[Add module unit tests here after MIS. —SS]

5.2.1 Module ?

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module ?

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

Initial State:

Input/Condition:

Output/Result:

How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Author Author. System requirements specification. https://github.com/..., 2019.

Canadian Engineering Accreditation Board. Accreditation Criteria and Procedures 2025. Engineers Canada, Ottawa, ON, Canada, November 2025. URL https://engineerscanada.ca/sites/default/files/2025-12/2025-Accreditation_Criteria_Procedures-EN.pdf. Accessed: Oct. 18, 2026.

McMaster University Senate. Undergraduate Course Management Policy. McMaster University, Hamilton, ON, Canada, May 2025. URL https:

//secretariat.mcmaster.ca/university-policies-procedures-guidelines/undergraduate-course-management-policy/undergraduate-course-management-policy-html/. Accessed: Oct. 18, 2026.

S. Mehta, A. Bornomala, H. S. Sagar, and V. Lai. Hazard Analysis. Team 19 (BVSH), Dept. of Computing and Software, McMaster University, Hamilton, ON, Canada, October 2026a. URL https://github.com/shivyam/cas-course-optimization/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf. Accessed: Nov. 1, 2026.

S. Mehta, A. Bornomala, H. S. Sagar, and V. Lai. Software Requirements Specification: CAS Course Deadline Management System. Team 19 (BVSH), Dept. of Computing and Software, McMaster University, Hamilton, ON, Canada, October 2026b. URL https://github.com/shivyam/cas-course-optimization/blob/main/pdfs/SRS-IEEE/SRS.pdf. Accessed: Oct. 19, 2026.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The system tests use these constants. Values match the SRS auxiliary constants where applicable.

- Cluster window = 7 days
- Cluster threshold = 3
- Min alternatives = 2
- Weight total = 100%
- Max outline size = 5 MB (`SSR_AcceptedFiles`)

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?